

Projet : Système de Gestion des Congés

Rapport Final — Gestion de Projet Agile (SCRUM)

EILCO
Année Universitaire 2025

Équipe de Développement

Scrum Master : Yahya CHARAF
Product Owner : Iliass AFKIR
Développeurs : Hamza BOUDA, Chaimae SEYYAR, Abderrahim OUAHAB
Testeuse : Aya BOUNOUA

Encadré par : Pr. GUALI Taha

Table des matières

Introduction	3
1 Présentation de l'Application	4
1.1 Contexte et objectif général	4
1.2 Rôles utilisateurs	5
1.3 Fonctionnalités principales	5
1.3.1 Visualisation des équipes	5
1.3.2 Gestion du profil collaborateur	6
1.3.3 Soumission d'une demande de congé	6
1.3.4 4. Notifications manager	7
1.3.5 5. Approbation / refus des demandes	8
1.3.6 6. Historique des demandes	8
1.4 Gestion Agile : Intégration des outils SCRUM	9
1.5 Synthèse fonctionnelle	9
2 Organisation SCRUM du Projet	10
2.1 Structure de l'équipe	10
2.2 Approche méthodologique	10
2.3 Gestion du Product Backlog	10
2.4 Les Cérémonies SCRUM	11
2.4.1 Sprint Planning	11
2.4.2 Daily Scrum	11
2.4.3 Sprint Review	11
2.4.4 Sprint Rétrospective	11
2.5 Suivi Kanban et Avancement	12
3 Synthèse des Sprints	13
3.1 Sprint 1 : Mise en place initiale et difficultés	13
3.2 Sprint 2 : Refonte complète et stabilisation	14
3.3 Sprint 3 : Gestion des demandes et notifications	14
3.4 Sprint 4 : Historique et amélioration du suivi	14
3.5 Sprint 5 : Stabilisation et dernières fonctionnalités	15
4 Rôle du Product Owner et Gestion du Backlog	16
4.1 Mon Rôle en tant que Product Owner (Iliass AFKIR)	16
4.2 Construction et Priorisation du Product Backlog	16
4.2.1 Construction	16
4.2.2 Priorisation	16
4.3 Élicitation et Clarification des User Stories	17

4.4	Critères d'Acceptation (Definition of Done)	17
4.5	Difficultés et Solutions	17
4.5.1	Les difficultés rencontrées	17
4.5.2	Ce que j'ai amélioré	18
4.6	Difficultés et Adaptation du Product Owner	18
4.6.1	Synthèse des Difficultés (Inspection)	18
4.6.2	Actions Correctives et Amélioration Continue (Adaptation)	19
5	Rôle de l'Équipe de Développement	20
5.1	Mission et Philosophie de l'Équipe	20
5.2	Évolution de l'Architecture Technique	20
5.2.1	L'impasse du Sprint 1 : La complexité face à l'urgence	20
5.2.2	Le pivot vers une architecture Hybride (Django + React)	21
5.3	Répartition des Rôles et Contributions	21
5.3.1	Hamza BOUDA : Architecture de données et Logique métier	21
5.3.2	Chaimae SEYYAR : Chef d'orchestre des flux	22
5.3.3	Abderrahim OUAHAB : Expérience Utilisateur (Frontend)	22
5.4	Défis Techniques et Solutions Apportées	22
5.5	Bilan de l'Équipe	23
6	Rôle du Testeur et Assurance Qualité	24
6.1	Mon Rôle en tant que Testeuse	24
6.1.1	Responsabilités principales	24
6.2	Collaboration avec l'équipe SCRUM	24
6.2.1	Daily Scrum	24
6.2.2	Sprint Planning	25
6.2.3	Sprint Review	25
6.2.4	Sprint Rétrospective	25
6.3	Plans de Test par Sprint	25
6.3.1	Sprint 1 : Mise en place du projet	25
6.3.2	Sprint 2 : Visualisation des équipes et demande de congé	26
6.3.3	Sprint 3 : Gestion complète des demandes	27
6.3.4	Sprint 4 : Historique et modification des demandes	29
6.3.5	Sprint 5 : Débogage et amélioration finale	30
6.4	Synthèse globale de la campagne de tests	31
6.4.1	Difficultés rencontrées	31
6.4.2	Actions d'amélioration mises en place	32
6.5	Bilan personnel sur le rôle de Testeur	32
6.5.1	Compétences développées	32
6.5.2	Valeur ajoutée au projet	32
6.5.3	Apprentissages clés	33
6.5.4	Difficultés surmontées	33
6.5.5	Conclusion personnelle	33
	Conclusion	34

Introduction

Dans le cadre du module Méthodes Agiles , notre équipe a développé une application web complète intitulée **Système de Gestion des Congés**. Ce projet a été conduit en suivant la méthode **SCRUM**, permettant d'organiser efficacement le travail d'équipe, d'assurer une livraison incrémentale et d'intégrer régulièrement les retours du client. L'objectif

principal de l'application est d'offrir une solution centralisée pour la gestion des demandes de congés au sein d'une organisation. Le système permet aux collaborateurs de soumettre leurs demandes, aux managers de les examiner, de les approuver ou de les refuser, tout en maintenant un suivi clair des soldes, des historiques et des notifications. L'interface a été conçue afin de garantir une expérience fluide, intuitive et cohérente entre les différents rôles utilisateurs. Tout au long du projet, cinq sprints successifs ont été réalisés, chacun

visant à livrer un incrément fonctionnel : visualisation des équipes, demande de congé, notifications, gestion des décisions du manager, historique des demandes et corrections finales. Grâce à cette démarche Agile, l'équipe a pu améliorer progressivement la solution, corriger rapidement les obstacles rencontrés et atteindre un **MVP stable, fonctionnel et évolutif**. Le présent rapport regroupe l'ensemble du travail réalisé durant le projet :

organisation de l'équipe, gestion du backlog, suivi des sprints, résultats obtenus, ainsi que les contributions techniques, fonctionnelles et qualitatives des différents membres. Il constitue la synthèse finale du travail collaboratif effectué selon les principes et les valeurs du cadre SCRUM.

Chapitre 1

Présentation de l'Application

1.1 Contexte et objectif général

Le **Système de Gestion des Congés** est une application web permettant d'automatiser et de simplifier le processus de gestion des demandes de congés au sein d'une organisation. Elle assure une meilleure transparence, une communication plus fluide entre collaborateurs et managers, ainsi qu'une traçabilité complète des décisions.

L'ensemble du développement a été réalisé en suivant la méthodologie **Scrum**, avec une évolution incrémentale sur cinq sprints successifs.

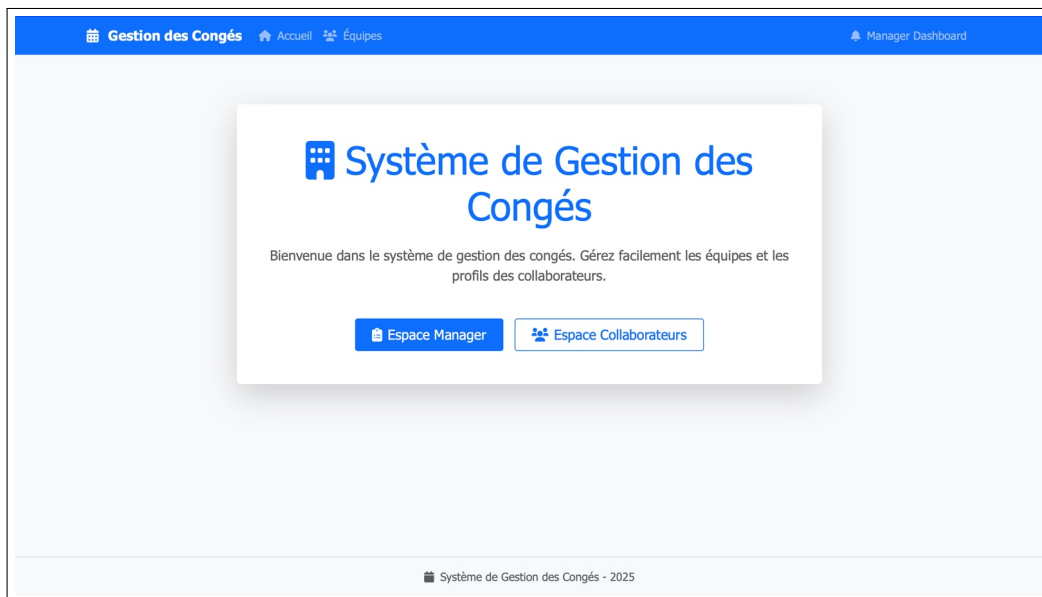


FIGURE 1.1 – Interface d'accueil de l'application

1.2 Rôles utilisateurs

L'application distingue plusieurs profils :

- **Collaborateur** : gestion de son profil, dépôt de demandes, suivi des soldes et historique.
- **Manager** : validation ou refus des demandes, consultation des équipes, accès aux historiques.

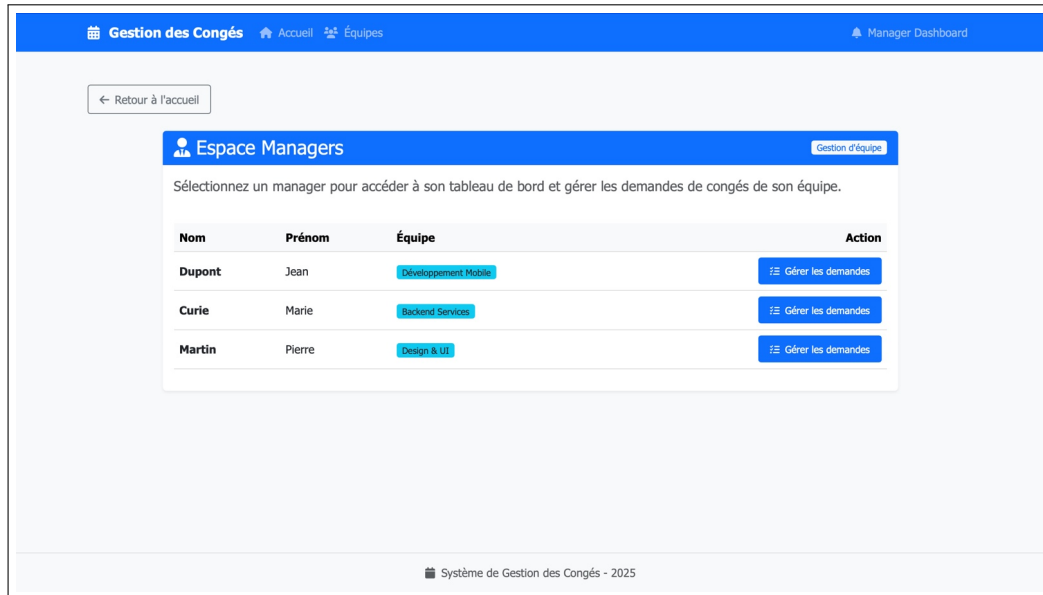


FIGURE 1.2 – Exemple d'interface du profil Manager

1.3 Fonctionnalités principales

L'application inclut un ensemble de fonctionnalités majeures développées progressivement à travers les sprints.

1.3.1 Visualisation des équipes

Le manager dispose d'un tableau affichant les membres de son équipe et leurs informations pertinentes (projet, localisation, soldes de congés, etc.).

← Retour à la liste des managers

Équipe de Jean Dupont

Membres de l'équipe

Nom du collaborateur	Projet	Localisation	Congés Épuisés	Congés Restants
Jean Dupont	Développement Mobile	Paris	0,5 jours	31,5 jours
Sophie Bernard	Développement Mobile	Paris	0,0 jours	32,0 jours
Lucas Petit	Développement Mobile	Paris	0,0 jours	32,0 jours
Camille Leroy	Développement Mobile	Paris	0,0 jours	32,0 jours

Demandes de congés en attente

Collaborateur	Période	Jours	Commentaire	Actions
Aucune demande en attente				

FIGURE 1.3 – Visualisation des équipes

1.3.2 Gestion du profil collaborateur

Chaque collaborateur accède à son espace personnel contenant :

- ses informations personnelles,
- ses soldes de congés,
- un bouton pour soumettre une demande,
- un bouton pour consulter son historique.

← Retour à la liste

Mon Profil Collaborateur

Informations personnelles

Nom Dupont	Prénom Jean
Date d'embauche 01/01/2020	Équipe Développement Mobile

Soldes de Congés

Congés restants : 31.5 jours Congés épuisés : 0.5 jours

Gestion des Congés

Vous pouvez initier une demande de congé ou consulter vos demandes existantes.

[Poser une demande](#) [Mes Demandes](#)

FIGURE 1.4 – Exemple d'interface du profil collaborateur

1.3.3 Soumission d'une demande de congé

Le formulaire de demande comprend :

- dates de début et de fin,

- motif,
- calcul automatique du nombre de jours,
- vérification des dates invalides,
- gestion des demi-journées.

The screenshot shows a web interface for submitting a new leave request. At the top, there's a blue header with 'Gestion des Congés', 'Accueil', 'Équipes', and 'Manager Dashboard'. Below the header, a button 'Retour au profil' is visible. The main form is titled 'Nouvelle Demande de Congé' and contains the following fields:

- Employee Info:** Jean Dupont, Équipe: Développement Mobile, Solde disponible: 31,5 jours.
- Request Type:** ☐ Demi-journée (0.5 jour).
- Dates:** Date de début: 24/11/2025, Date de fin: 24/11/2025.
- Motif (Optionnel):** A text area for 'Motif de la demande, informations complémentaires...'. Below it are 'Annuler' and 'Soumettre la demande' buttons.

At the bottom, a footer reads 'Système de Gestion des Congés - 2025'.

FIGURE 1.5 – Interface de soumission d’une demande de congé

1.3.4 4. Notifications manager

Lorsqu’un collaborateur soumet une demande, le manager est notifié dans un tableau de bord dédié, avec compteur dynamique.

The screenshot shows the 'Manager Dashboard' with a notification modal open. The modal is titled 'Demandes en attente' and displays a table of pending requests:

Collaborateur	Période	Statut
Jean Dupont	18/12/2025 - 20/12/2025	EN COURS

Below the modal, there's a table showing the leave status for the team 'Équipe de Jean Dupont':

Nom du collaborateur	Projet	Localisation	Congés Épuisés	Congés Restants
Jean Dupont	Développement Mobile	Paris	0,5 jours	31,5 jours
Sophie Bernard	Développement Mobile	Paris	0,0 jours	32,0 jours
Lucas Petit	Développement Mobile	Paris	0,0 jours	32,0 jours
Camille Leroy	Développement Mobile	Paris	0,0 jours	32,0 jours

At the bottom, there's a section 'Demandes de congés en attente' with a table showing pending requests:

Collaborateur	Période	Jours	Commentaire	Actions
Jean Dupont	Du 18/12/2025 Au 20/12/2025	3,0		☒ Motif ☐

FIGURE 1.6 – Exemple d’affichage des notifications manager (Sprint 3)

1.3.5 5. Approbation / refus des demandes

Le manager peut :

- approuver une demande,
- refuser une demande avec un motif obligatoire,
- consulter les détails de la demande.

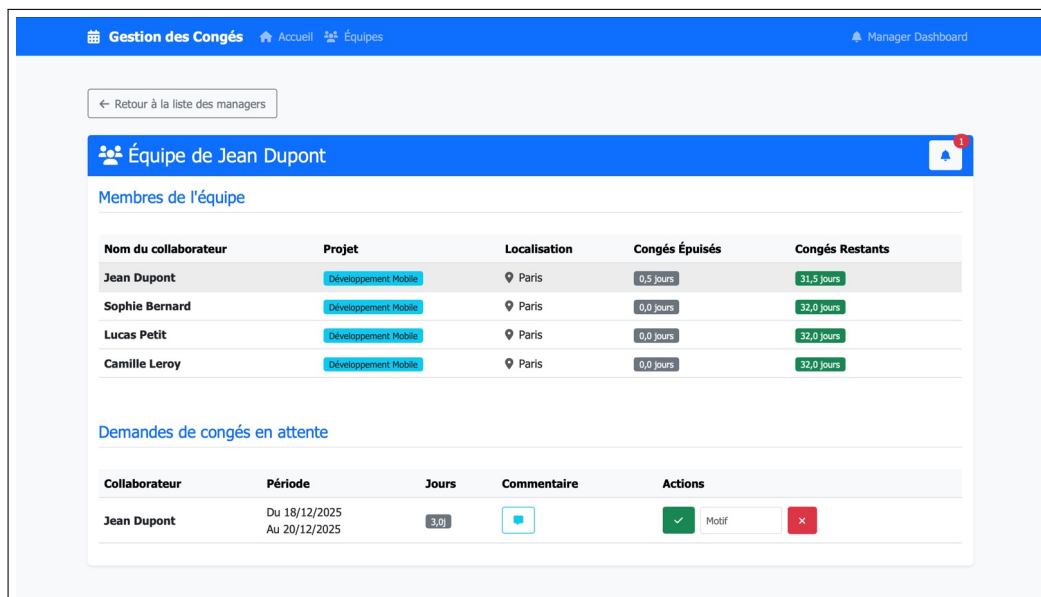


FIGURE 1.7 – Interface d’approbation/refus des demandes

1.3.6 6. Historique des demandes

Deux interfaces ont été développées :

- historique personnel du collaborateur ;
- historique global du manager.

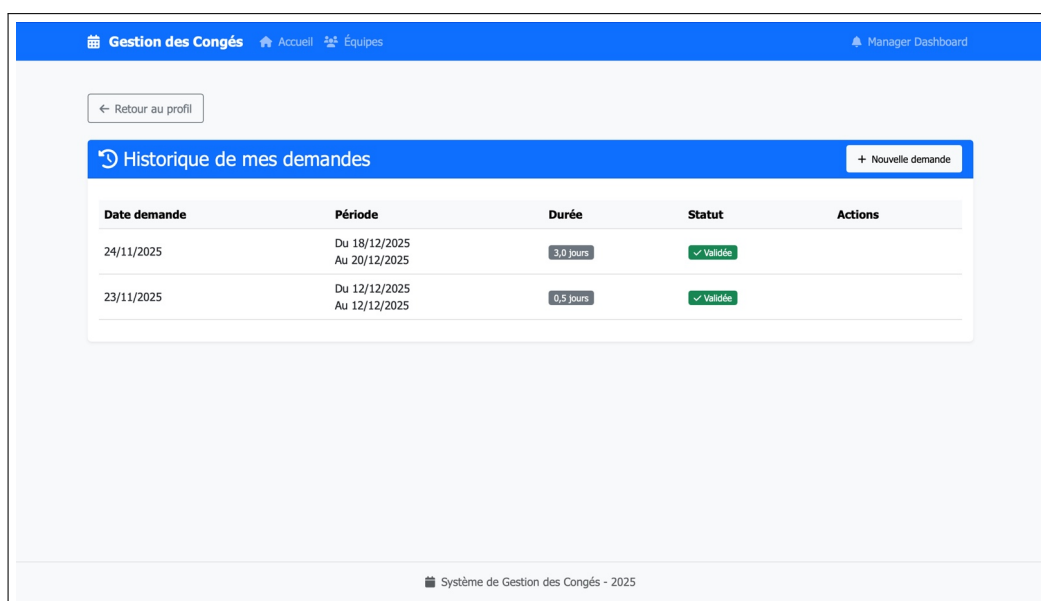


FIGURE 1.8 – Historique des demandes

1.4 Gestion Agile : Intégration des outils SCRUM

Au cours du développement, des outils professionnels ont été utilisés :

- **Jira Software** pour gérer le backlog, les user stories, les sous-tâches et le tableau Kanban.
- **GitHub** pour le versionnement, les branches et les revues de code.

Les captures ci-dessous montrent une partie du backlog et du tableau utilisé durant les sprints.

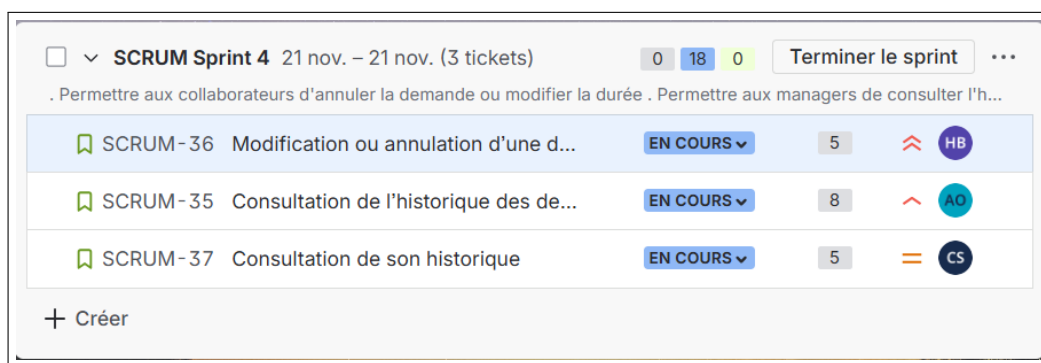


FIGURE 1.9 – Backlog Jira (user stories et sous-tâches)

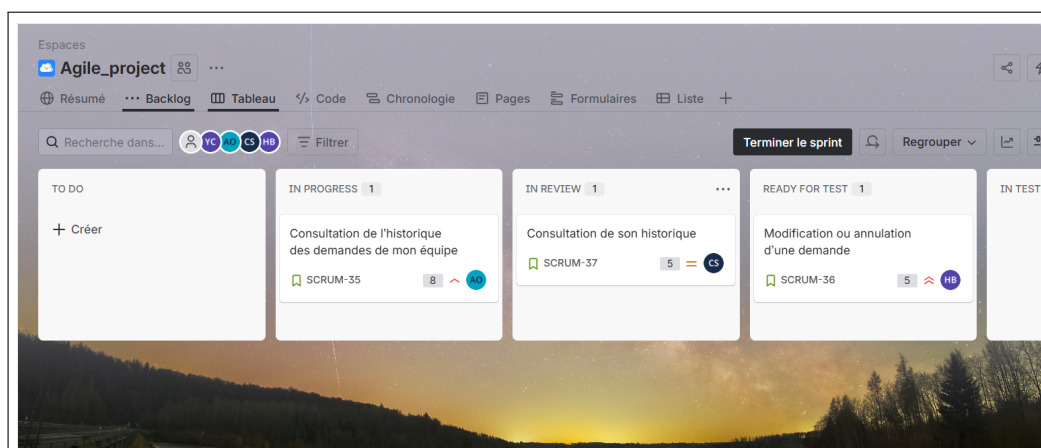


FIGURE 1.10 – Tableau Jira (To Do / In Progress / Done)

1.5 Synthèse fonctionnelle

L'application propose un système complet, évolutif et conforme aux besoins d'une organisation moderne, offrant :

- une gestion fluide des congés,
- un suivi rigoureux,
- une interface claire pour chaque rôle,
- une traçabilité complète des actions.

Les interfaces présentées dans ce chapitre témoignent de l'évolution progressive du projet et de la cohérence des fonctionnalités développées au fil des sprints.

Chapitre 2

Organisation SCRUM du Projet

2.1 Structure de l'équipe

Le développement de l'application a été conduit selon la méthodologie Scrum, avec une équipe composée de six membres ayant chacun un rôle clairement défini :

- **Scrum Master** : Yahya CHARAF Responsable de l'animation des cérémonies, de la facilitation et du respect du cadre méthodologique.
- **Product Owner** : Iliass AFKIR Membre interne de l'équipe chargé de représenter les besoins du client, de définir et prioriser le Product Backlog, et de garantir la valeur fonctionnelle du produit.
- **Développeurs** : Hamza BOUDA, Chaimae SEYYAR, Abderrahim OUAHAB Responsables du développement des fonctionnalités, de l'architecture technique et des intégrations.
- **Testeur** : Aya BOUNOUA Chargée de la rédaction des scripts de tests, de la validation fonctionnelle et de la détection des anomalies.

2.2 Approche méthodologique

L'équipe a appliqué la méthodologie Scrum afin d'assurer un développement structuré, itératif et collaboratif. Les trois piliers du cadre **transparence**, **inspection** et **adaptation** ont guidé la progression tout au long du projet.

Les échanges réguliers entre le Product Owner, l'équipe de développement et le Scrum Master ont permis :

- une compréhension partagée des priorités,
- une progression incrémentale du produit,
- une amélioration continue des pratiques d'équipe,
- une qualité fonctionnelle stable des livrables.

2.3 Gestion du Product Backlog

Le **Product Backlog** a été géré sous la responsabilité du Product Owner, qui assurait :

- la rédaction et la clarification des user stories,
- la définition des critères d'acceptation,
- la priorisation des fonctionnalités,

- l’ajustement des éléments selon les besoins exprimés par le client.

Les user stories ont ensuite été décomposées en tâches techniques par les développeurs, puis estimées collectivement à l’aide de la méthode du *Planning Poker*.

Jira Software a été utilisé pour organiser le backlog et pour assurer une traçabilité claire de l’évolution des tâches.

2.4 Les Cérémonies SCRUM

2.4.1 Sprint Planning

Chaque sprint débutait par une séance de planification durant laquelle :

- l’objectif du sprint était fixé,
- les user stories prioritaires étaient présentées par le Product Owner,
- les tâches techniques étaient identifiées et réparties,
- l’équipe estimait collectivement la charge de travail.

2.4.2 Daily Scrum

Étant donné que chaque sprint correspondait à une séance de travail de deux heures, la **Daily Scrum** était organisée à la fin de la séance. Elle permettait à l’équipe de faire un point clair sur l’avancement réel du sprint :

- ce qui a été réalisé au cours de la séance,
- ce qui n’a pas pu être terminé,
- les difficultés rencontrées,
- les éléments à prendre en considération pour le sprint suivant.

Cette réunion courte offrait une vision précise de la progression de chaque membre et permettait d’anticiper les ajustements nécessaires pour les prochaines itérations.

2.4.3 Sprint Review

La **Sprint Review** était réalisée en fin de séance. L’équipe y présentait l’incrément développé au Product Owner et au client. Cette étape permettait de :

- démontrer les fonctionnalités ajoutées,
- valider les user stories terminées,
- recueillir les retours fonctionnels,
- ajuster certaines priorités si nécessaire.

2.4.4 Sprint Rétrospective

Juste après la Review, la rétrospective du sprint était menée afin d’améliorer la collaboration et l’efficacité de l’équipe. Elle permettait d’identifier :

- les points positifs,
- les difficultés rencontrées,
- les actions d’amélioration à appliquer dès le sprint suivant.

2.5 Suivi Kanban et Avancement

Le tableau **Kanban Jira** a servi d'outil principal pour visualiser l'avancement des tâches tout au long du sprint. Il était structuré en six colonnes permettant de représenter précisément le cycle de développement :

- **To Do** : tâches planifiées en attente de traitement
- **In Progress** : tâches en cours de développement
- **In Review** : tâches terminées en attente de revue par un autre membre
- **Ready for Test** : tâches prêtes à être testées par le testeur
- **In Test** : tâches en phase de vérification fonctionnelle
- **Done** : tâches terminées, validées et intégrées

Ce découpage précis a permis une meilleure visibilité sur le flux de travail, une coordination fluide entre développeurs et testeur, ainsi qu'un suivi clair du statut de chaque tâche.

Chapitre 3

Synthèse des Sprints

Ce chapitre retrace les différentes itérations réalisées durant le développement du *Système de Gestion des Congés*. Chaque sprint correspond à une séance de travail de deux heures incluant : planification, développement, Daily en fin de séance, revue, et rétrospective.

3.1 Sprint 1 : Mise en place initiale et difficultés

Objectif du sprint

- Mettre en place une première architecture fonctionnelle.
- Afficher un tableau des équipes.
- Connecter l'interface à la base de données.

Travaux réalisés

- Base de données initiale.
- Première ébauche d'affichage des équipes.
- Début de développement du backend.

Difficultés rencontrées

Le sprint 1 n'a pas atteint son objectif. Les principales difficultés observées étaient :

- manque d'harmonisation entre backend et frontend,
- architecture trop lourde et peu flexible,
- répartition des tâches non optimale,
- manque de communication sur les dépendances techniques.

Bilan

L'incrément produit n'était pas exploitable. La rétrospective a conduit à une décision majeure : **refaire l'architecture dans le Sprint 2 et repartir sur des bases plus solides.**

3.2 Sprint 2 : Refonte complète et stabilisation

Objectif du sprint

- Reconcevoir l'architecture technique.
- Refaire entièrement l'écran de visualisation des équipes.
- Ajouter l'espace collaborateur + demande de congé.

Travaux réalisés

- Refonte complète du backend et du frontend.
- Nouvelle interface de visualisation des équipes.
- Développement de la page profil collaborateur.
- Première version fonctionnelle de la demande de congé.

Bilan

Le sprint 2 a posé les bases stables du projet. Les flux collaborateur–manager ont commencé à émerger correctement, permettant d'aborder sereinement les fonctionnalités avancées des sprints suivants.

3.3 Sprint 3 : Gestion des demandes et notifications

Objectif

- Finaliser le flux de demande de congé.
- Mettre en place les notifications manager.
- Implémenter l'approbation et le refus avec motif.

Travaux réalisés

- Formulaire complet de demande.
- Notification en temps réel côté manager.
- Approbation/refus + motif obligatoire.
- Tests fonctionnels dédiés.

Bilan

Le flux collaborateur → manager est devenu entièrement opérationnel. Le sprint a marqué une étape clé de la montée en valeur du produit.

3.4 Sprint 4 : Historique et amélioration du suivi

Objectif

- Implémenter l'historique collaborateur.
- Implémenter l'historique manager.
- Ajouter modification et annulation des demandes.

Travaux réalisés

- Interface d’historisation complète.
- Historique manager détaillé.
- Actions “modifier” et “annuler”.

Bilan

Le sprint 4 a permis d’atteindre un **MVP complet et cohérent**. Le système couvre désormais tout le cycle de vie d’une demande.

3.5 Sprint 5 : Stabilisation et dernières fonctionnalités

Objectif

- Corriger les anomalies signalées.
- Ajouter la fonctionnalité de demi-journée.

Travaux réalisés

- Correction du calcul dynamique.
- Blocage des dates incorrectes.
- Refonte du comportement du formulaire.
- Ajout de la demi-journée.

Bilan

Le sprint 5 finalise une version stable, testée et prête à la démonstration, intégrant toutes les attentes du client.

Chapitre 4

Rôle du Product Owner et Gestion du Backlog

4.1 Mon Rôle en tant que Product Owner (Iliass AF-KIR)

En tant que Product Owner (PO), ma mission était d'assurer que l'équipe développe le bon produit, celui qui apporte le plus de valeur au client. J'étais le représentant du client au sein de l'équipe de développement.

Mes principales tâches étaient :

- Définir la vision du produit (le Système de Gestion des Congés).
- Gérer et prioriser le Product Backlog pour que l'équipe travaille toujours sur ce qui est le plus important.
- Clarifier les User Stories auprès de l'équipe pour qu'ils comprennent bien le besoin fonctionnel.
- Valider l'incrément à la fin de chaque Sprint pour m'assurer que les fonctionnalités correspondent aux attentes.

C'était un rôle central pour lier le besoin client et le travail technique de l'équipe.

4.2 Construction et Priorisation du Product Backlog

Le Product Backlog est la liste de tout ce qu'il faut faire pour le projet. Ma responsabilité était de le construire et surtout de le maintenir à jour.

4.2.1 Construction

J'ai commencé par la liste des besoins principaux, comme "pouvoir demander un congé" ou "voir l'historique". J'ai rédigé tout ça sous forme de User Stories (histoire utilisateur), car c'est plus facile à comprendre pour tout le monde.

4.2.2 Priorisation

La priorité était donnée aux fonctionnalités qui apportaient le plus de valeur et qui étaient essentielles pour avoir un Minimum Viable Product (MVP) rapide. Par exemple,

faire la demande de congé (Sprint 2 et 3) était prioritaire sur des fonctions secondaires. Cette priorisation était revue avant chaque *Sprint Planning*.

4.3 Élicitation et Clarification des User Stories

Une User Story doit être claire. J'ai veillé à ce qu'elles suivent le format standard :

En tant que [rôle utilisateur], je souhaite [objectif] afin de [bénéfice].

Par exemple :

- *En tant que Collaborateur, je souhaite pouvoir soumettre une demande de congé afin que mon manager puisse la valider.*
- *En tant que Manager, je souhaite recevoir une notification pour toute nouvelle demande afin de pouvoir la traiter rapidement.*

Pendant le *Sprint Planning* ou l'affinage du backlog, je prenais le temps d'expliquer chaque story à l'équipe. Ça permettait aux développeurs de poser des questions et d'estimer correctement la difficulté (Planning Poker).

4.4 Critères d'Acceptation (Definition of Done)

Pour moi, en tant que PO, un élément du Backlog n'était "terminé" (Done) que s'il respectait les Critères d'Acceptation.

Ces critères sont des points précis qui décrivent ce que la fonctionnalité doit faire pour être considérée comme réussie et validée. Par exemple, pour la story "Soumettre une demande de congé", les critères d'acceptation étaient :

- L'utilisateur doit pouvoir sélectionner une date de début et de fin.
- Le système doit calculer automatiquement le nombre de jours ouvrés.
- Une notification doit être envoyée au manager désigné.
- Le solde de congé du collaborateur doit être mis à jour après la validation du manager.

Ces critères étaient importants pour Aya (la testeuse) car ils servaient de base pour écrire ses scénarios de tests. En gros, si tous les critères d'acceptation étaient OK, je pouvais valider la fonctionnalité lors de la Sprint Review.

4.5 Difficultés et Solutions

Ce projet m'a permis de comprendre que le rôle de Product Owner ne s'arrête pas à l'écriture de tickets sur Jira. J'ai rencontré plusieurs obstacles qu'il a fallu surmonter avec l'équipe.

4.5.1 Les difficultés rencontrées

- **Des débuts difficiles (Sprint 1) :** Au lancement du projet, la communication avec l'équipe de développement n'était pas assez fluide. Mes instructions n'étaient pas toujours bien comprises et les tickets manquaient parfois de détails clairs sur ce qui était attendu. Résultat : le premier incrément n'était pas totalement satisfaisant.

- **Changements de priorités (Sprint 3) :** J’ai dû apprendre à être flexible. Par exemple, suite à un changement de besoin exprimé par le client, j’ai dû retirer une tâche de correction de bug du backlog pour prioriser trois nouvelles User Stories urgentes (Saisie, Notification, Approbation). C’était une décision stressante car il fallait refaire le planning en cours de route.
- **Nouvelles demandes tardives (Sprint 5) :** Alors qu’on pensait avoir fini avec le MVP, le client a demandé l’ajout de la gestion des **demi-journées** après avoir testé l’application. J’ai dû intégrer cette fonctionnalité au dernier moment dans le sprint final.

4.5.2 Ce que j’ai amélioré

Pour régler ces soucis, j’ai mis en place des actions correctives :

- J’ai instauré des points de clarification rapides avec les développeurs pour valider la compréhension des tâches avant le lancement du code.
- J’ai pris l’habitude de rappeler les fonctionnalités globales à chaque début de Sprint pour que tout le monde garde la même vision du produit.
- J’ai mieux géré le backlog pour atteindre le MVP (Minimum Viable Product) dès la fin du Sprint 4.

4.6 Difficultés et Adaptation du Product Owner

Le rôle de Product Owner, étant nouveau pour moi, a nécessité une phase d’apprentissage rapide. Le projet a été marqué par des obstacles qui, grâce à l’agilité de l’équipe, ont été transformés en opportunités d’amélioration continue.

4.6.1 Synthèse des Difficultés (Inspection)

Les principaux problèmes rencontrés, surtout lors des premiers sprints, étaient :

- **Prise en main des outils :** Un manque d’habitude avec **Jira** au début (Sprint 1) a rendu difficile la manipulation et la rédaction des tickets. Je ne connaissais pas les meilleures pratiques pour les structurer, ce qui a pu engendrer un flou dans le Product Backlog.
- **Définition des besoins et négociation client :** J’ai eu du mal à obtenir une définition complète du besoin. Il m’a fallu apprendre à poser les bonnes questions pour tirer tous les points fonctionnels du client et établir des **Critères d’Acceptation** suffisamment précis pour l’équipe de développement et le testeur.
- **Problèmes de communication (Sprint 1) :** En conséquence du manque de clarté des tickets, la communication avec les développeurs était insuffisante. Mes instructions n’étaient pas toujours bien comprises, ce qui a ralenti la production du premier incrément.
- **Gestion des changements de priorités (Sprint 3) :** J’ai été confronté à la pression de devoir retirer une tâche de correction de bug du backlog pour intégrer et prioriser immédiatement trois User Stories critiques (Saisie, Notification, Approbation) suite à un besoin client urgent.
- **Demande tardive du client (Sprint 5) :** Une fois le MVP atteint, le client a fait un retour demandant l’ajout de la fonctionnalité de gestion des **demi-journées**.

J'ai dû intégrer cette nouvelle fonctionnalité au dernier sprint, qui était initialement prévu uniquement pour le débogage.

4.6.2 Actions Correctives et Amélioration Continue (Adaptation)

Ces difficultés initiales ont été rapidement corrigées en appliquant les principes du cadre Scrum, notamment l'adaptation suite aux *Rétrospectives*.

- **Maîtrise de Jira et du Backlog** : J'ai rapidement pris en main l'outil pour rédiger des tickets plus propres, complets et mieux structurés (avec les User Stories claires et les critères d'acceptation détaillés).
- **Amélioration de la communication** : J'ai instauré des points de clarification rapides avec les développeurs pour valider la compréhension des tâches avant le lancement du code. J'ai aussi pris l'habitude de rappeler la vision globale du produit au début de chaque Sprint.
- **Priorisation stricte** : J'ai appris à dire non aux fonctionnalités à faible valeur ajoutée et à mieux gérer l'ordre du backlog pour m'assurer que l'équipe se concentrait toujours sur les tâches les plus importantes, ce qui a permis d'atteindre le MVP dès la fin du Sprint 4.
- **Anticipation des critères** : J'ai systématisé la validation des critères d'acceptation avec le client pour chaque User Story, assurant une meilleure traçabilité et facilitant le travail de la testeuse.

L'ensemble de ce processus d'amélioration, de l'inspection des difficultés à l'adaptation des pratiques, a été clé dans la réussite finale du projet.

Bilan personnel sur le rôle de Product Owner

Pour conclure sur mon rôle, cette expérience en tant que Product Owner a été extrêmement bénéfique. Elle m'a permis de sortir du cadre purement théorique pour me confronter à la réalité du terrain et aux défis de la gestion de projet Agile.

J'ai beaucoup apprécié cet exercice pratique qui m'a aidé à renforcer mes connaissances en les appliquant concrètement. C'était une véritable opportunité de transformer les concepts vus en cours en compétences opérationnelles, en apprenant à gérer une équipe, un backlog et les attentes d'un client dans un contexte réel.

Chapitre 5

Rôle de l'Équipe de Développement

5.1 Mission et Philosophie de l'Équipe

L'équipe de développement, constituée de Hamza BOUDA, Chaimae SEYYAR et Abderrahim OUAHAB, a représenté le moteur technique du projet "Système de Gestion des Congés". Notre responsabilité dépassait la simple rédaction de lignes de code : nous étions garants de la faisabilité technique des demandes du client et de la robustesse de la solution finale.

Dans le cadre de la méthode Scrum, nous avons dû changer notre approche habituelle de travail. Plutôt que de travailler isolément sur des tâches techniques, nous avons adopté une vision "orientée produit". Cela signifie que pour chaque ligne de code écrite, nous nous posions la question : *"Cela apporte-t-il de la valeur immédiate à l'utilisateur final ?"*

Notre mission s'est articulée autour de trois axes majeurs :

- **La co-conception** : Nous avons collaboré étroitement avec le Product Owner pour transformer des besoins fonctionnels parfois abstraits en solutions techniques concrètes et réalisables.
- **L'engagement sur les délais** : Lors des *Sprint Plannings*, nous avons appris à estimer notre propre vélocité pour ne promettre que ce que nous étions capables de livrer en 2 heures.
- **L'amélioration continue** : Chaque bug remonté par notre testeuse (Aya) n'était pas vu comme un échec, mais comme une opportunité de sécuriser l'application avant la livraison finale.

5.2 Évolution de l'Architecture Technique

L'histoire technique de ce projet est marquée par une remise en question profonde survenue au tout début du cycle de développement. Ce pivot stratégique a été déterminant pour la réussite du projet.

5.2.1 L'impasse du Sprint 1 : La complexité face à l'urgence

Au lancement du projet, animés par la volonté d'utiliser les technologies les plus populaires du marché, nous sommes partis sur une architecture dite "Headless" (séparée) : un backend en Java SpringBoot et un frontend en React.js, communiquant via une API REST.

Cependant, la réalité des sprints courts (sessions de 2 heures) nous a rapidement rattrapés. Durant le Sprint 1, nous avons consacré 80% de notre temps à la configuration technique : faire "discuter" le serveur Java avec l'interface React, gérer les problèmes de sécurité (CORS), et configurer les environnements de développement sur nos machines respectives.

Le résultat fut sans appel : à la fin du premier sprint, nous avions une architecture théoriquement puissante, mais aucune fonctionnalité concrète à montrer au client. L'interface ne pouvait pas afficher les données correctement. Nous avons compris que la complexité de cette architecture n'était pas adaptée à la contrainte de temps du projet.

5.2.2 Le pivot vers une architecture Hybride (Django + React)

Lors de la rétrospective du Sprint 1, l'équipe a pris une décision courageuse : changer de technologie backend pour gagner en productivité, tout en conservant la modernité de l'interface. Nous avons opté pour le couple Django (Python) et React.js, mais intégrés de manière astucieuse.

Pourquoi ce choix a-t-il sauvé le projet ?

1. **La puissance de Django pour le Backend :** Contrairement à SpringBoot qui demande beaucoup de configuration initiale, Django suit la philosophie "Batteries Included". Il nous a offert immédiatement un système d'administration, une gestion des utilisateurs et une connexion simplifiée à la base de données. Cela nous a permis de nous concentrer sur le code métier (les congés) plutôt que sur la "tuyauterie" technique.
2. **L'intégration simplifiée de React :** Au lieu de faire tourner deux serveurs séparés qui peinent à communiquer, nous avons intégré React directement à l'intérieur des "Vues" de Django. En résumé, Django gère la sécurité et les données, et il "sert" la page React à l'utilisateur. Cette architecture hybride a éliminé les problèmes de connexion réseau complexes. Nous avons pu profiter de la fluidité visuelle de React (pas de rechargement de page, composants interactifs) sans subir la lourdeur de gestion d'une API complexe.

5.3 Répartition des Rôles et Contributions

Pour être efficaces, nous avons divisé le travail en couches techniques, tout en maintenant une communication constante pour assurer que les différentes parties s'emboîtent parfaitement.

5.3.1 Hamza BOUDA : Architecture de données et Logique métier

Je me suis concentré sur la fondation invisible mais essentielle de l'application, c'est-à-dire le "Cerveau" du système :

- **Modélisation des données :** J'ai structuré la base de données pour qu'elle puisse stocker intelligemment les collaborateurs, les équipes et les demandes, en utilisant l'ORM de Django (qui permet de manipuler la base de données comme du code Python simple).

- **Règles de gestion** : J'ai programmé les contrôles de sécurité, comme l'interdiction de poser des congés dans le passé ou la vérification que la date de fin est bien après la date de début.
- **Calculs complexes** : J'ai développé l'algorithme qui calcule automatiquement la durée du congé en excluant les week-ends, et adapté ce calcul au Sprint 5 pour gérer les cas particuliers des demi-journées (0,5 jour).

5.3.2 Chaimae SEYYAR : Chef d'orchestre des flux

Mon rôle se situait à l'intersection entre les données brutes et l'interface utilisateur. Je devais m'assurer que l'information circule sans encombre.

- **Communication Backend-Frontend** : J'ai développé les mécanismes (Vues Django) qui préparent les données issues de la base (ex : liste des congés) et les envoient au format correct (JSON) pour que React puisse les afficher.
- **Gestion des Notifications** : J'ai conçu la logique d'alerte. Lorsqu'une demande est créée, le système doit instantanément identifier le manager responsable et "marquer" l'information pour qu'une notification apparaisse sur son tableau de bord.
- **Traitement des actions** : J'ai géré le cycle de vie des demandes. Quand un manager clique sur "Valider", mon code s'assure que le statut change en base de données et que le solde du collaborateur est débité immédiatement.

5.3.3 Abderrahim OUAHAB : Expérience Utilisateur (Frontend)

Je me suis concentré sur la partie visible de l'application, celle avec laquelle l'utilisateur interagit quotidiennement.

- **Interface Réactive avec React** : J'ai construit les pages de l'application sous forme de composants modulaires. Cela permet une navigation fluide : quand on change d'onglet, seule la partie nécessaire de la page se met à jour, sans rechargement complet.
- **Ergonomie Manager** : J'ai travaillé spécifiquement sur le tableau de bord du manager pour le rendre efficace. L'objectif était qu'un manager puisse voir en un coup d'œil qui est absent et valider une demande en moins de deux clics.
- **Design et Retours visuels** : J'ai veillé à ce que l'utilisateur soit toujours guidé (messages de succès en vert, erreurs en rouge, champs obligatoires mis en évidence) pour éviter les frustrations.

5.4 Défis Techniques et Solutions Apportées

Au-delà du changement d'architecture, le développement a été parsemé d'obstacles que nous avons dû surmonter collectivement.

1. Le casse-tête des Dates

Travailler avec des dates est notoirement difficile en informatique. Nous avons rencontré des problèmes où une date sélectionnée le "12 Mars" dans l'interface React arrivait comme "11 Mars à 23h00" dans la base de données Django à cause des fuseaux horaires.
Solution : Nous avons dû standardiser rigoureusement le format des dates (ISO 8601) à

travers toute l'application pour garantir qu'il n'y ait aucun décalage entre ce que l'utilisateur voit et ce qui est enregistré.

2. La collaboration sur le code (Git)

Travailler à trois sur les mêmes fichiers a souvent provoqué des conflits de code (quand deux développeurs modifient la même fonction en même temps). Au début, cela nous faisait perdre du temps à "réparer" le projet.

Solution : Nous avons instauré une discipline plus stricte. Nous avons travaillé sur des branches séparées et nous faisons des sessions de "code review" ensemble avant de fusionner nos travaux, ce qui a réduit drastiquement les bugs de régression.

3. Le défi de dernière minute : Les demi-journées

Au Sprint 5, l'ajout des demi-journées a semblé simple fonctionnellement mais a été complexe techniquement. Notre base de données était conçue pour compter des jours entiers (nombres entiers).

Solution : Il a fallu modifier la structure de la base de données pour accepter des nombres décimaux (Type Float) et réécrire toute la logique de calcul de solde pour gérer les cas comme "0.5 jours". Ce fut un sprint intense de refactoring (réécriture de code) pour ne pas casser l'existant.

5.5 Bilan de l'Équipe

Cette expérience nous a transformés. Nous avons appris qu'en méthode Agile, **la capacité d'adaptation est plus importante que le plan initial**.

Le passage de SpringBoot à Django/React nous a enseigné le pragmatisme : il vaut mieux une technologie simple que l'on maîtrise et qui permet de livrer vite, qu'une technologie complexe qui nous ralentit. De plus, la collaboration constante avec le Product Owner et la Testeuse nous a sortis de notre isolement technique pour nous focaliser sur la qualité réelle du produit livré.

Nous sommes fiers d'avoir livré, malgré les difficultés de départ, une application stable, moderne et parfaitement fonctionnelle.

Chapitre 6

Rôle du Testeur et Assurance Qualité

6.1 Mon Rôle en tant que Testeuse

En tant que Testeuse au sein de l'équipe Scrum, ma mission était d'assurer la qualité fonctionnelle du produit développé tout au long des sprints. Mon travail consistait à vérifier que chaque fonctionnalité livrée correspondait bien aux attentes définies par le Product Owner et respectait les critères d'acceptation établis.

6.1.1 Responsabilités principales

Mes principales responsabilités comprenaient :

- **Rédaction des scénarios de test** : Conception de cas de test couvrant les fonctionnalités développées, basés sur les user stories et les critères d'acceptation.
- **Exécution des tests fonctionnels** : Validation manuelle de chaque fonctionnalité livrée en fin de sprint pour s'assurer de sa conformité.
- **Détection et signalement des anomalies** : Identification des bugs et dysfonctionnements, documentation précise dans Jira et suivi jusqu'à leur résolution.
- **Validation des corrections** : Vérification systématique de la résolution effective des anomalies signalées lors des sprints précédents.

Mon objectif était de garantir que l'incrément livré à chaque fin de sprint soit stable, fonctionnel et conforme aux attentes du client.

6.2 Collaboration avec l'équipe SCRUM

En alignement avec la méthodologie agile, j'ai participé activement aux cérémonies Scrum pour assurer une intégration continue de la qualité dans le processus de développement.

6.2.1 Daily Scrum

Lors des réunions quotidiennes, je partageais :

- Le suivi des anomalies en cours de correction.
- L'état d'avancement des tests prévus : tests réussis, échoués ou bloqués.
- Les obstacles rencontrés nécessitant l'intervention de l'équipe ou du Scrum Master.

Cette communication régulière permettait une réactivité immédiate face aux problèmes de qualité.

6.2.2 Sprint Planning

Durant la planification de sprint, mon rôle consistait à :

- Analyser les user stories prévues pour le sprint afin de comprendre les fonctionnalités à tester.
- Estimer le temps de test nécessaire pour chaque fonctionnalité.
- Identifier les prérequis pour les campagnes de tests (données de test, environnements, accès).
- Collaborer avec les développeurs .

6.2.3 Sprint Review

À la fin de chaque sprint, je présentais :

- Les résultats de tests réalisés pendant le sprint (nombre de tests exécutés, taux de réussite).
- La validation ou le rejet des fonctionnalités en fonction de leur conformité aux critères d'acceptation.
- Les anomalies critiques qui bloquent la validation de certaines user stories.

Cette présentation permettait au Product Owner et au client de prendre des décisions éclairées sur l'acceptation de l'incrément.

6.2.4 Sprint Rétrospective

Lors des rétrospectives, je contribuais à l'amélioration continue en :

- Identifiant les points d'amélioration concernant la coordination entre les tests et le développement.
- Proposant des ajustements pour renforcer la qualité des versions suivantes (tests plus précoces, meilleure documentation, etc.).
- Partageant les bonnes pratiques observées durant le sprint.

6.3 Plans de Test par Sprint

Cette section présente les plans de test réalisés pour chaque sprint, incluant les critères d'acceptation, les scénarios de test, les résultats obtenus et la validation finale de chaque fonctionnalité.

6.3.1 Sprint 1 : Mise en place du projet

Fonctionnalité testée : Affichage de l'équipe

Critères d'acceptation :

- Le manager doit pouvoir visualiser la liste des membres de son équipe.
- Les informations affichées doivent inclure : nom, prénom, poste, projet, localisation.
- Les données doivent être récupérées depuis la base de données.

Scénarios de test :

ID	Scénario	Résultat attendu	Résultat	Statut
T1.1	Accéder à la page d’affichage de l’équipe en tant que manager	La liste des membres s’affiche correctement	Liste affichée avec données incomplètes	Non validé
T1.2	Vérifier que toutes les colonnes attendues sont présentes	Colonnes : Nom, Prénom, Poste, Projet, Localisation	Colonne Localisation manquante	Non validé
T1.3	Vérifier la cohérence des données affichées avec la base de données	Données conformes à la BDD	Quelques incohérences détectées	Non validé

TABLE 6.1 – Résultats des tests - Sprint 1

Bilan Sprint 1 : Fonctionnalité **NON VALIDÉE**. Plusieurs anomalies ont été identifiées et remontées à l’équipe de développement pour correction lors du sprint suivant.

6.3.2 Sprint 2 : Visualisation des équipes et demande de congé

Fonctionnalité 1 : Tableau d’affichage des équipes

Critères d’acceptation :

- Le tableau doit afficher toutes les informations pertinentes des membres.
- Les données doivent être à jour et complètes.
- L’interface doit être claire et lisible.

Scénarios de test :

ID	Scénario	Résultat attendu	Résultat	Statut
T2.1	Accéder au tableau des équipes	Affichage complet et cohérent	Conforme	Validé
T2.2	Vérifier l’affichage des soldes de congés	Soldes visibles pour chaque membre	Conforme	Validé
T2.3	Tester l’affichage avec une équipe vide	Message approprié ou tableau vide	Conforme	Validé

TABLE 6.2 – Résultats des tests - Tableau des équipes (Sprint 2)

Bilan : Fonctionnalité **VALIDÉE**.

Fonctionnalité 2 : Page profil collaborateur

Critères d’acceptation :

- Le collaborateur doit accéder à son profil personnel.
- Les informations personnelles et les soldes de congés doivent être affichés.
- la soumission de demande doit être présent et fonctionnel.

Scénarios de test :

ID	Scénario	Résultat attendu	Résultat	Statut
T2.4	Connexion en tant que collaborateur et accès au profil	Profil affiché avec toutes les informations	Conforme	Validé
T2.5	Vérifier l’affichage des soldes de congés	Solde restant visible et correct	Conforme	Validé
T2.6	Cliquer sur le bouton "Demander un congé"	Redirection vers le formulaire de demande	Conforme	Validé

TABLE 6.3 – Résultats des tests - Profil collaborateur (Sprint 2)

Bilan : Fonctionnalité **VALIDÉE**.

Fonctionnalité 3 : Soumission d’une demande de congé

Critères d’acceptation :

- Le formulaire doit permettre la saisie des dates de début et de fin.
- Le motif de la demande doit être obligatoire.
- Le système doit calculer automatiquement le nombre de jours ouvrés.
- Une notification doit être envoyée au manager après soumission.

Scénarios de test :

ID	Scénario	Résultat attendu	Résultat	Statut
T2.7	Saisir une demande avec dates valides et motif	Demande soumise avec succès	Calcul de jours incorrect	Non validé
T2.8	Soumettre sans motif	Message d’erreur	Pas de message d’erreur	Non validé
T2.9	Saisir une date de fin antérieure à la date de début	Message d’erreur	Validation incorrecte	Non validé

TABLE 6.4 – Résultats des tests - Demande de congé (Sprint 2)

Bilan : Fonctionnalité **NON VALIDÉE**. Anomalies remontées pour correction au Sprint 3.

6.3.3 Sprint 3 : Gestion complète des demandes

Fonctionnalité 1 : Formulaire de saisie de demande (corrections)

Critères d’acceptation :

- Calcul automatique et correct du nombre de jours ouvrés.
- Validation stricte des dates (date de fin > date de début).
- Champ motif obligatoire avec message d’erreur si vide.

Scénarios de test :

ID	Scénario	Résultat attendu	Résultat	Statut
T3.1	Saisir demande avec dates valides	Calcul correct du nombre de jours	Conforme	Validé
T3.2	Soumettre sans motif	Message d'erreur affiché	Conforme	Validé
T3.3	Date de fin antérieure à date de début	Blocage de la soumission	Conforme	Validé
T3.4	Tester avec un week-end inclus	Exclusion correcte des week-ends	Conforme	Validé

TABLE 6.5 – Résultats des tests - Formulaire corrigé (Sprint 3)

Bilan : Fonctionnalité **VALIDÉE**.

Fonctionnalité 2 : Système de notifications manager

Critères d'acceptation :

- Le manager reçoit une notification pour chaque nouvelle demande.
- Un compteur dynamique affiche le nombre de demandes en attente.
- Les notifications sont cliquables et redirigent vers les détails de la demande.

Scénarios de test :

ID	Scénario	Résultat attendu	Résultat	Statut
T3.5	Soumettre une demande en tant que collaborateur	Notification visible côté manager	Conforme	Validé
T3.6	Vérifier le compteur de notifications	Incrémentation correcte du compteur	Conforme	Validé
T3.7	Cliquer sur une notification	Redirection vers détails de la demande	Conforme	Validé
T3.8	Vérifier l'actualisation après traitement	Compteur décrémenté après validation/refus	Conforme	Validé

TABLE 6.6 – Résultats des tests - Notifications (Sprint 3)

Bilan : Fonctionnalité **VALIDÉE**.

Fonctionnalité 3 : Approbation et refus des demandes

Critères d'acceptation :

- Le manager peut approuver une demande.
- Le manager peut refuser une demande avec un motif obligatoire.
- Le statut de la demande est mis à jour après chaque action.
- Le collaborateur est notifié de la décision.

Scénarios de test :

ID	Scénario	Résultat attendu	Résultat	Statut
T3.9	Approuver une demande en attente	Statut = "Approuvée", solde mis à jour	Conforme	Validé
T3.10	Refuser une demande sans motif	Message d'erreur : motif obligatoire	Conforme	Validé
T3.11	Refuser une demande avec motif	Statut = "Refusée", motif enregistré	Conforme	Validé
T3.12	Vérifier la notification collaborateur après approbation	Notification reçue côté collaborateur	Conforme	Validé

TABLE 6.7 – Résultats des tests - Approbation/Refus (Sprint 3)

Bilan : Fonctionnalité **VALIDÉE**.

6.3.4 Sprint 4 : Historique et modification des demandes

Fonctionnalité 1 : Historique collaborateur

Critères d'acceptation :

- Le collaborateur peut consulter l'historique de toutes ses demandes.
- Les informations affichées incluent : dates, motif, statut, décision du manager.
- L'historique est trié par ordre chronologique décroissant.

Scénarios de test :

ID	Scénario	Résultat attendu	Résultat	Statut
T4.1	Accéder à l'historique en tant que collaborateur	Liste complète des demandes passées	Conforme	Validé
T4.2	Vérifier le tri chronologique	Demandes triées du plus récent au plus ancien	Conforme	Validé
T4.3	Vérifier l'affichage des statuts	Statuts corrects (Approuvée, Refusée, En attente)	Conforme	Validé
T4.4	Tester avec un historique vide	Message approprié affiché	Conforme	Validé

TABLE 6.8 – Résultats des tests - Historique collaborateur (Sprint 4)

Bilan : Fonctionnalité **VALIDÉE**.

Fonctionnalité 2 : Historique manager

Critères d'acceptation :

- Le manager peut consulter l'historique de toutes les demandes de son équipe.
- Possibilité de filtrer par collaborateur ou par statut.
- Affichage des décisions prises et des motifs de refus.

Scénarios de test :

ID	Scénario	Résultat attendu	Résultat	Statut
T4.5	Accéder à l'historique global en tant que manager	Liste de toutes les demandes de l'équipe	Conforme	Validé
T4.6	Filtrer par nom de collaborateur	Affichage uniquement des demandes du collaborateur sélectionné	Conforme	Validé
T4.7	Filtrer par statut (Approuvée)	Affichage uniquement des demandes approuvées	Conforme	Validé
T4.8	Vérifier l'affichage des motifs de refus	Motifs visibles pour les demandes refusées	Conforme	Validé

TABLE 6.9 – Résultats des tests - Historique manager (Sprint 4)

Bilan : Fonctionnalité **VALIDÉE**.

Fonctionnalité 3 : Modification et annulation des demandes

Critères d'acceptation :

- Le collaborateur peut modifier une demande en attente.
- Le collaborateur peut annuler une demande en attente.
- Les demandes déjà approuvées ou refusées ne peuvent pas être modifiées.

Scénarios de test :

ID	Scénario	Résultat attendu	Résultat	Statut
T4.9	Modifier une demande en attente	Formulaire pré-rempli, modification enregistrée	Conforme	Validé
T4.10	Annuler une demande en attente	Demande supprimée de la liste	Conforme	Validé
T4.11	Tenter de modifier une demande approuvée	Action bloquée avec message d'information	Conforme	Validé
T4.12	Tenter d'annuler une demande refusée	Action bloquée avec message d'information	Conforme	Validé

TABLE 6.10 – Résultats des tests - Modification/Annulation (Sprint 4)

Bilan : Fonctionnalité **VALIDÉE**.

Bilan global Sprint 4 : Le MVP est atteint. Toutes les fonctionnalités majeures sont opérationnelles et validées.

6.3.5 Sprint 5 : Débogage et amélioration finale

Fonctionnalité 1 : Corrections des bugs identifiés

Critères d'acceptation :

- Correction de l'insertion dynamique des dates.
- Blocage effectif des dates invalides (passées, incohérentes).
- Stabilisation du calcul automatique de la durée.

Scénarios de test :

ID	Scénario	Résultat attendu	Résultat	Statut
T5.1	Saisir une date passée	Blocage avec message d'erreur	Conforme	Validé
T5.2	Tester le calcul avec différentes périodes	Calcul toujours correct	Conforme	Validé
T5.3	Vérifier l'insertion dynamique des dates	Dates correctement insérées dans la BDD	Conforme	Validé

TABLE 6.11 – Résultats des tests - Corrections (Sprint 5)

Bilan : Corrections **VALIDÉES**.

Fonctionnalité 2 : Gestion des demi-journées

Critères d'acceptation :

- Le collaborateur peut sélectionner l'option "Demi-journée".
- Le calcul du nombre de jours prend en compte les demi-journées (0,5 jour).
- Le solde de congés est correctement mis à jour après validation.

Scénarios de test :

ID	Scénario	Résultat attendu	Résultat	Statut
T5.4	Demander une demi-journée	Calcul = 0,5 jour	Conforme	Validé
T5.5	Vérifier la mise à jour du solde après approbation	Solde décrétement de 0,5 jour	Conforme	Validé
T5.6	Combiner demi-journée et jours entiers	Calcul correct du total	Conforme	Validé

TABLE 6.12 – Résultats des tests - Demi-journées (Sprint 5)

Bilan : Fonctionnalité **VALIDÉE**.

Bilan global Sprint 5 : Application stable, conforme aux attentes et prête pour la livraison finale.

6.4 Synthèse globale de la campagne de tests

6.4.1 Difficultés rencontrées

Durant ma mission de test, j'ai été confrontée à plusieurs défis :

- **Sprint 1 : Manque de clarté des critères d'acceptation** Les user stories initiales manquaient parfois de précision, ce qui rendait difficile la rédaction de scénarios de test exhaustifs. J'ai dû solliciter régulièrement le Product Owner pour clarifier les attentes.
- **Sprint 2 : Environnement de test instable** Des problèmes de connexion à la base de données ont retardé l'exécution de certains tests. Une meilleure coordination avec l'équipe de développement a permis de résoudre ce problème rapidement.
- **Sprint 3 : Volume important de fonctionnalités** Trois fonctionnalités majeures ont été livrées simultanément, nécessitant une grande charge de travail de test dans un temps limité. J'ai dû prioriser les tests critiques.

- **Sprint 5 : Ajout tardif d'une fonctionnalité** L'ajout de la gestion des demi-journées en fin de projet a nécessité la rédaction rapide de nouveaux scénarios de test et leur exécution dans un délai court.

6.4.2 Actions d'amélioration mises en place

Pour surmonter ces difficultés, j'ai mis en œuvre plusieurs actions correctives :

- **Participation active au Sprint Planning** J'ai renforcé ma présence lors des planifications pour clarifier immédiatement les critères d'acceptation avec le Product Owner, permettant une rédaction plus précise des scénarios de test dès le début du sprint.
- **Communication proactive avec les développeurs** J'ai instauré des échanges réguliers avec l'équipe de développement pour signaler rapidement les anomalies et suivre leur correction en temps réel, accélérant ainsi le cycle de feedback.
- **Documentation détaillée des anomalies** J'ai systématisé la documentation des bugs dans Jira avec captures d'écran, étapes de reproduction et niveau de criticité, facilitant leur compréhension et leur résolution par les développeurs. .

6.5 Bilan personnel sur le rôle de Testeur

Cette expérience en tant que testeuse dans un projet Agile a été extrêmement enrichissante et formatrice. Elle m'a permis de comprendre l'importance cruciale de la qualité dans le cycle de développement logiciel et le rôle central du testeur dans une équipe Scrum.

6.5.1 Compétences développées

Au cours de ce projet, j'ai pu développer plusieurs compétences essentielles :

- **Rédaction de scénarios de test** : J'ai appris à transformer des critères d'acceptation en cas de test concrets, couvrant à la fois les scénarios nominaux et les cas limites.
- **Détection et analyse des anomalies** : J'ai développé un sens critique pour identifier rapidement les dysfonctionnements et évaluer leur impact sur l'expérience utilisateur.
- **Communication technique** : J'ai appris à documenter clairement les bugs et à communiquer efficacement avec les développeurs pour faciliter leur résolution.
- **Gestion des outils Agiles** : La maîtrise de Jira pour le suivi des anomalies et l'organisation des campagnes de test.

6.5.2 Valeur ajoutée au projet

Mon travail de testeuse a contribué significativement à la qualité finale du produit :

- **Détection précoce des anomalies** : les bugs majeurs ont été identifiés et corrigés avant la livraison finale, évitant ainsi des problèmes en production.
- **Amélioration de la qualité fonctionnelle** : Les tests systématiques ont permis de garantir que chaque fonctionnalité livrée respectait les attentes du Product Owner.
- **Feedback constructif** : Mes retours réguliers ont aidé l'équipe à améliorer progressivement la qualité du code et des interfaces.

- **Confiance du client** : La rigueur des tests a rassuré le client sur la fiabilité du système livré.

6.5.3 Apprentissages clés

Cette expérience m’a permis de tirer plusieurs enseignements importants :

- **Le test est une activité collaborative** : Contrairement à ce que je pensais initialement, le testeur ne travaille pas en isolation. La collaboration étroite avec les développeurs, le Product Owner et le Scrum Master est essentielle pour garantir la qualité.
- **L’importance de la clarté des critères d’acceptation** : Des critères flous rendent les tests inefficaces. J’ai appris à solliciter des précisions dès le Sprint Planning pour éviter les ambiguïtés.
- **Tester tôt et souvent** : Attendre la fin du développement pour tester est une erreur. Les tests continus tout au long du sprint permettent de corriger rapidement et à moindre coût.
- **L’équilibre entre rigueur et pragmatisme** : Il n’est pas toujours possible de tout tester. J’ai appris à prioriser les tests en fonction des risques et de la valeur métier.

6.5.4 Difficultés surmontées

Au-delà des difficultés techniques mentionnées précédemment, j’ai également dû surmonter des défis personnels :

- **Gestion du temps** : Apprendre à estimer correctement le temps nécessaire pour chaque campagne de test a été un défi, surtout lors du Sprint 3 où le volume était important.
- **Communication assertive** : J’ai dû apprendre à défendre mes positions concernant la qualité, même lorsque cela impliquait de retarder une livraison pour corriger un bug critique.
- **Adaptation aux changements** : L’ajout tardif de fonctionnalités (comme les demi-journées au Sprint 5) m’a obligée à être flexible et réactive.

6.5.5 Conclusion personnelle

Cette expérience de testeuse dans un environnement Agile a été une opportunité précieuse pour mettre en pratique les concepts théoriques vus en cours. J’ai particulièrement apprécié :

- La dimension collaborative du travail en équipe Scrum.
- Le sentiment de contribution directe à la qualité du produit final.
- L’apprentissage continu à travers les rétrospectives et les échanges avec l’équipe.
- La satisfaction de voir les anomalies détectées être corrigées et le produit s’améliorer sprint après sprint.

Ce projet m’a confortée dans l’idée que l’assurance qualité est un métier passionnant, essentiel et valorisant dans le développement logiciel moderne. Je suis convaincue que les compétences acquises durant ce projet me seront extrêmement utiles dans ma future carrière professionnelle.

Conclusion

Le développement du *Système de Gestion des Congés* a permis à l'équipe de mettre en pratique de manière concrète les principes du cadre Scrum et de construire progressivement une application complète répondant aux besoins exprimés par le client. À travers les cinq sprints réalisés, l'équipe a su s'adapter, corriger ses approches, améliorer sa collaboration et renforcer la qualité de ses livrables. Les itérations successives ont conduit à un

produit final stable et fonctionnel, intégrant l'essentiel des fonctionnalités nécessaires au cycle de gestion des demandes de congés : visualisation des équipes, espace collaborateur, soumission de demandes, système de notifications, traitement manager, historique complet et prise en compte des demi-journées. Chaque sprint a apporté une valeur ajoutée mesurable, tout en permettant une amélioration continue du processus de travail. Au-delà

de l'aspect technique, ce projet a été une occasion d'expérimenter de manière réaliste le travail en équipe dans un cadre Agile : organisation collective, communication, synchronisation, gestion des priorités et adaptation aux retours du client. L'expérience acquise au fil des sprints renforce la capacité de l'équipe à mener des projets collaboratifs plus ambitieux et plus structurés. En définitive, le projet atteint ses objectifs, tant sur le plan

fonctionnel qu'organisationnel. La démarche itérative adoptée a permis de livrer un produit cohérent, testé et aligné avec les attentes du client, tout en offrant à l'équipe une compréhension approfondie des enjeux et exigences d'un développement Agile réussi.